Android API Field Evolution and Its Induced Compatibility Issues

Tarek Mahmud
Texas State University
USA
t_m386@txstate.edu

Meiru Che
Data61, CSIRO
Australia
meiruche@gmail.com

Guowei Yang*
The University of Queensland
Australia
guowei.yang@uq.edu.au

ABSTRACT

Background: The continuous evolution of the Android operating system necessitates regular API updates, which may affect the functionality of Android apps. Recent studies investigated API evolution to ensure the reliability of Android apps; however, they focused on API methods alone. **Aim:** We aim to empirically investigate how Android API fields evolve, and how this evolution affects the compatibility of Android apps. **Method:** We conducted a study based on real-world app development history data involving 11098 tags out of 105 popular open-source Android apps. **Results:** Our study yields interesting findings, e.g., on average two API field compatibility issues exist per app, different types of checks are preferred when addressing different types of compatibility issues, and fixing compatibility issues induced by API field evolution takes more time than fixing compatibility issues induced by API method evolution. **Conclusion:** These findings will help developers and researchers better understand, detect, and handle Android compatibility issues induced by API field evolution.

CCS CONCEPTS

• **General and reference** → **Empirical studies**; • **Software and its engineering** → **Software evolution**; *Maintaining software.*

KEYWORDS

Android, compatibility issues, API evolution, API fields

ACM Reference Format:
Tarek Mahmud, Meiru Che, and Guowei Yang. 2022. Android API Field Evolution and Its Induced Compatibility Issues. In *ACM / IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM) (ESEM '22), September 19–23, 2022, Helsinki, Finland.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3544902.3546242

1 INTRODUCTION

Mobile devices have cemented their importance and become a necessary component in many aspects of daily lives, such as navigation, social media, education, and multi-player online strategy games. *Android* is the most widely used mobile operating system, representing over 70% of the market share [11]. In 2017, it achieved a major milestone of having over two billion monthly active devices [41]. Developers are constantly creating new apps for Android. According to *Google Play Store* [9], the number of available apps reached 2.8 million in 2020 [44]. Android apps are developed using a Software Development Kit (SDK), where the Android application programming interface (API) enables app developers to harness the functionalities of Android devices by interacting with services and hardware. However, APIs are frequently updated for Android OS, from API level 1 in the first commercial release of Android 1.0 on September 23rd, 2008 [2] to the current API level 30. In each update, they introduced new fields and methods as well as removed or made changes to the existing fields and methods.

Android app developers have limited control over the devices where their apps will run. Thus, Google recommends that developers specify the range of the API levels their apps can support in the manifest or Gradle file by setting *minSdkVersion* and *maxSdkVersion*, in addition to specifying the target API level during app development. However, developers may not verify or validate their apps on all the API levels in such range. The mismatch between the API level supported by the device where apps are actually run and the API level targeted by developers can induce compatibility issues, which may manifest themselves as unexpected behaviors, including runtime crashes, creating a poor user experience [47].

Researchers have recently conducted several studies on the evolution of Android ecosystem [29, 32–35, 39, 49, 50] that lead to several techniques for detecting and handling API-related compatibility issues in Android apps [26, 27, 30, 36, 47, 48]. However, these techniques either target at API methods alone or may require extra modifications to work on API fields. For example, Li et al. proposed CiD [30], which uses Android Lifecycle Modeling (ALM) and Android Usage Extraction (AUE) to detect API method invocation compatibility issues. He et al. [26] proposed IctApiFinder, which computes the OS versions on which an API can be invoked using inter-procedural data-flow analysis but cannot be generalized to API fields according to the paper. To the best of our knowledge, the evolution of Android API fields (i.e., public fields of an API class) has rarely been studied before. Thus, a study on Android API field evolution is necessary to understand how similar or different API [1] field evolution is compared to API method evolution, and how to deal with API field evolution where they are different.

This paper presents an empirical study on the evolution of Android API fields, and its induced compatibility issues, based on real-world app development history. Our goal is to find out how API fields evolve, and how API field evolution affects API compatibility. Moreover, we are also interested in how this type of

*Corresponding author.

ESEM '22, September 19–23, 2022, Helsinki, Finland

ACM ISBN 978-1-4503-9427-7/22/09…$15.00
https://doi.org/10.1145/3544902.3546242

[1] All occurrences of "API" refer to "Android API" by default in the rest of this paper unless the scope of the "API" is explicitly mentioned.

compatibility issues are handled, and how much time is taken to fix these issues. To achieve this, we investigated 11098 tags out of 105 popular open-source Android apps. In addition, we need to know the number of compatibility issues induced by API field evolution; however, there is no technique available for automated detection of such issues. To facilitate our study, we introduce a novel technique for automated detection of such issues in Android apps.

This paper yields interesting findings about Android API field evolution and its induced compatibility issues. For example,

- API fields evolve frequently, and its evolution rate is only 12.14% less than API method evolution (when compatibility relevant evolution is considered).
- On average, there are two API field compatibility issues per app in each tag.
- The use of "@RequiresApi" is surprisingly high in addressing API field compatibility issues. The "if-then" and "if-then-else" API level checks are more popular when addressing forward compatibility issues, while "@RequiresApi" is more popular when addressing backward compatibility issues.
- On average, it takes more than three months to fix API field compatibility issues.

We believe the findings of our empirical study will help developers and researchers better understand, detect and handle compatibility issues induced by Android API field evolution.

This paper makes the following contributions:

- **Study.** An empirical study of Android API field evolution and its induced compatibility issues in real-world Android apps, with a detailed investigation on 11098 tags of 105 open-source Android apps.
- **Technique.** A novel technique for automatically detecting compatibility issues induced by Android API field evolution.
- **Implications.** Multiple implications for researchers and developers in advancing knowledge and practices on Android API field evolution.

2 BACKGROUND ON API COMPATIBILITY ISSUES

Android App developers are recommended to specify a range of API levels that they officially endorse. By providing specific values to three attributes, i.e., minSdkVersion, targetSdkVersion and maxSdkVersion, developers specify which API levels their app supports. Each new API level contains improvement that affects the code of the apps. This involves updated APIs (using new trends), deprecated APIs, new APIs (possibly replacing the deprecated ones), and APIs that have been removed (several of which have been deprecated for a few versions). Since Android apps can run on devices with different API levels, the mismatch between the API level supported by the device and the API level targeted by developers can induce compatibility issues. In particular, backward compatibility issues happen when an app is running on a device with a lower API level than the targetSdkVersion, and the app is using API fields or methods that were introduced as the API evolved. On the other hand, forward compatibility issues occur when an app is running on a device with a higher API level than the targetSdkVersion, and the app is using API fields or methods that have been deprecated or removed during API evolution.

Suppose an app is created with the values 21, 27, and 29 for minSdkVersion, targetSdkVersion, and maxSdkVersion, respectively. If the app runs on a device with API level 23 and uses API fields or methods that are available in API level 27 but not in API level 23, then it will cause backward compatibility issues. If the app runs on a device with API level 29 and uses API fields or methods that are available in API level 27 but not in API level 29, then it will cause forward compatibility issues. Not only the removed or added methods or fields can create compatibility issues, some changed methods or fields also induce compatibility issues, such as methods or fields identifier changed from non-final to final. If an API field is changed to final in an API level, it cannot be updated when running in devices with higher API levels. For API methods, it cannot be overridden if it is changed to final.

Listing 1 shows an example of API compatibility issue caused by an incompatible API field, which we took from `Omni Notes` [12] where the targetSdkVersion is 28 and the minSdkVersion is 18. It used a deprecated API field Settings.System.NEXT_ALARM_FORMATTED (line 16) which was removed in API level 21. This code was fixed using the SDK version check (line 3) in tag 4.7.1, which is 6 months after the release of API level 21. Before the fix was applied, if the app was installed on a device with API level 21 or above, it would crash on the invocation of Settings.System.getString method with the deprecated API field Settings.System.NEXT_ALARM_FORMATTED (lines 14 – 16).

In this paper, we use *API field compatibility issues* to refer to the compatibility issues induced by API field evolution, and *API method compatibility issues* to refer to the compatibility issues induced by API method evolution.

```
1 private void testAlarms() {
2    String nextAlarm = null;
3    if(android.os.Build.VERSION.SDK_INT >=
4     android.os.Build.VERSION_CODES.LOLLIPOP) {
5       AlarmManager am =
6       (AlarmManager)getSystemService(Context.ALARM_SERVICE);
7       am.getNextAlarmClock();
8       try {
9          nextAlarm = am.getNextAlarmClock().toString();
10      } catch (Exception e) {
11         e.printStackTrace();
12      }
13   } else {
14      nextAlarm = Settings.System.getString(
15         getContentResolver(),
16         Settings.System.NEXT_ALARM_FORMATTED);
17   }
18   if(TextUtils.isEmpty(nextAlarm)) {
19      hideAlarm();
20   } else {
21      showAlarm();
22   }
23 }
```

Listing 1: An example of API compatibility issues caused by an incompatible API field.

To avoid compatibility issues induced by API evolution, developers need to perform API level checks on the running platform before calling incompatible API methods or API fields, to ensure that newly added APIs are not invoked on previous API levels and deprecated APIs are not invoked on subsequent API levels. The

"if-then", "if-then-else" and "@RequiresApi" annotations are used to perform these checks.

In particular, the "if-then" check is an easy way to handle compatibility issues where developers check a particular API level against SDK_INT (which provides the API level of the running device) before invoking any incompatible API. The "if-then-else" check is a safer way to address evolution-induced compatibility problems by providing an alternate implementation/function that has similar functionalities with a different API. Listing 1 shows an example of "if-then-else" check where different implementations are available for different API levels. The annotations are also ways of API-level check where "@RequiresApi" [13] denotes that the annotated element should only be called on the given API level or higher. There is another annotation "@TargetApi" [14] indicating that Lint, [17], which is a plugin for Android that analyzes app code to find potential issues, should treat this form as targeting a given API level no matter what the project target is. Since it is only for removing errors from the Lint, we are not considering this one. Listing 2 is an example usage of "@RequiresApi".

```
@RequiresApi(28)
private static Bitmap decodeImageSource(
   ImageDecoder.Source imageSource, final Request request)
throws IOException {
   return ImageDecoder.decodeBitmap(imageSource,
         new ImageDecoder.OnHeaderDecodedListener() {
      @Override
      public void onHeaderDecoded(
               @NonNull ImageDecoder imageDecoder,
               @NonNull ImageDecoder.ImageInfo imageInfo,
               @NonNull ImageDecoder.Source source) {
         if (request.hasSize()) {
            Size size = imageInfo.getSize();
            if (shouldResize(request.onlyScaleDown,
                  size.getWidth(), size.getHeight(),
            request.targetWidth, request.targetHeight)) {
               imageDecoder.setTargetSize(
                  request.targetWidth,
                  request.targetHeight);
            }
         }
      }
   });
}
```

Listing 2: An example usage of @RequiresApi.

3 API FIELD COMPATIBILITY ISSUE DETECTION

To facilitate the empirical study, we need to know the number of compatibility issues in real-world apps; however, to our best knowledge, there is no tool available for automated detection of API field compatibility issues. In this work, we propose FCID, a novel approach to detecting API field compatibility issues called. This approach is motivated by our insights gained from manually analyzing some real-world API field compatibility issues, as well as ACID [36], a state-of-the-art tool for detecting API method compatibility issues.

3.1 Approach

Figure 1 shows the framework of FCID. It takes as input the source code of the app or the decompiled APK file and the Android API differences reports, and outputs API Field compatibility issues for each API level within the range of API levels that are supported by the app. FCID consists of three components: (1) API Difference Collector, (2) API Field Usage Locator, and (3) Compatibility Issue Detector.

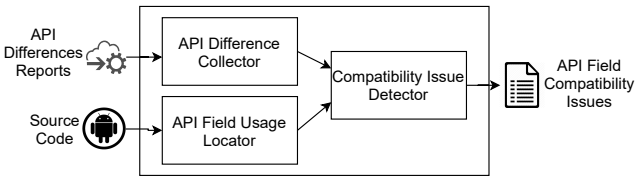

Figure 1: FCID overview.

3.1.1 API Difference Collector. This component takes as input API differences reports and the range of supported API levels, and outputs the incompatible API fields for each API level. We use suspicious API fields to refer to the API fields that could potentially cause compatibility issues if used in an app. This component first extracts the minSdkVersion, targetSdkVersion and maxSdkVersion from the app's build.gradle file [15]. However, developers usually do not specify maxSdkVersion for most apps, assuming they would support the highest available API level [5]. Therefore, we use 30 for maxSdkVersion if it is not explicitly specified, since the current highest API level on the market is 30. This component leverages the API differences report to find out all changes involved in an API update, and accumulates all the changes compared to the target API level to identify incompatible APIs.

Google publishes an API differences report every time a new level of Android API is published, summarizing the differences in the core Android API compared to its previous API level. The differences consist of additions, modifications, and removals for packages, classes, methods, and fields. For example, the summary of changes made from API 27 to API 28 can be found online at https://developer.android.com/sdk/api_diff/28/changes/changes-summary. To get a report on differences between an API level and its previous level, we need to modify the "28" in the URL to that API level.

The report's homepage consists of three tables: removed packages, added packages, and changed packages. Packages that have been removed and added are listed in the removed packages table and added packages table, respectively. The changed packages table lists all of the packages that were changed as part of the API upgrade. Each package in the table of changed packages has a link that takes to a class-level report of the classes in that package. It also has three tables of classes that have been added, changed and removed. The changed classes table, like the changed packages table, contains clickable links that lead to the method-level and field-level reports. This is where we collect information about API methods and fields that have been added, changed or removed. The table of added packages is also clickable, and it takes to the website for Android developer's documentation. Here is an example link https://developer.android.com/reference/android/app/role/package-summary.html, where the "android/app/role" is for the

package android.app.role. We get the information about added API fields here for each added package.

All of the packages that were removed from the old API are listed in the removed packages table. This table, unlike the added and changed packages table, lacks clickable links since all of the packages' classes, methods, and fields were also removed. However, the developer's documentation contains information on the methods and fields in these packages. For example, android.test.mock package is removed in the API level 28. However, the details of this package are available in the documentation webpage https://developer.android.com/reference/android/test/mock/package-summary.html.

This component identifies API differences for each API level. Not all API differences can cause compatibility issues for all API levels, and thus only the API differences that can cause compatibility issues are considered suspicious. In particular, added API fields can cause backward compatibility issues, and thus are considered suspicious for lower API levels where they were not introduced yet. Removed API fields can cause forward compatibility issues and thus they are considered suspicious for higher API levels where they are no longer existent. Here we do not consider the API fields with their identifier changed, for example, from non-final to final.

3.1.2 API Field Usage Locator. This component takes the app code as input and outputs all API field usages within the app. To locate the API field usages in the app code, this component generates program dependence graph of the app code to obtain the intra- and inter-procedural slices. Even if an Android app has used suspicious Android API fields that may cause API compatibility issues, we cannot report it before we check whether it is protected by condition checkers, which are used to prevent compatibility issues.

This component runs a backward data-flow analysis to check whether an API field is used under API level-related conditions. In practice, the conditions for getting access to an API field may not be easily identified. FCID uses backward data-flow analysis from the identified API field usages to check if it is used under any such conditions. If it is used under any API level checks, FCID keeps information of these constraints for compatibility issue detector.

For example, in the code of listing 1, this component first locates the API field usages, which are android.os.Build.VERSION.SDK_INT, android.os.Build.VERSION_CODES.LOLLIPOP, Context.ALARM_SERVICE and Settings.System.NEXT_ALARM_FORMATTED. For all these usages, FCID collects the information if those usages are covered by any API level conditions using backward data flow analysis from those usages. Here, Context.ALARM_SERVICE is under API level check greater that or equal to 22 (API level of Android version named Lollipop) and Settings.System.NEXT_ALARM_FORMATTED is under API level check less than 22.

3.1.3 Compatibility Issue Detector. Finally, the Compatibility Issue Detector checks which API field usages in the source code matches the suspicious API fields collected by API Difference Collector. If any of the suspicious API fields appears in the API field usages and the API field does not fulfill the constraint collected by backward data flow analysis, the API field usage will be marked as a compatibility issue.

Algorithm 1: Detecting Compatibility Issues Induced by API Field Evolution.

Input: Source Code $code$, Android API Differences Reports $APIDR$
Output: API field compatibility issues CI

```
   /* API Difference Collector                                   */
1  API_min ← getMinimumSdkVersionOfApp(Code)
2  API_target ← getTargetSdkVersionOfApp(Code)
3  API_max ← getMaximumSdkVersionOfApp(Code)
4  suspicious_fields := Φ
5  for level ∈ API_min to API_target do
6      suspicious_fields ← collectAddedFields(APIDR, level)
7  for level ∈ API_target to API_max do
8      suspicious_fields ←
         collectRemovedFields(APIDR, level)
   /* API Field Usage Locator                                    */
9  APIUsages ← collectAPIFIeldUsages(code)
   /* Compatibility Issue Detector                               */
10 for class ∈ APIUsages do
11     for method ∈ class do
12         for statement ∈ method do
13             for level ∈ suspicious_fields.keySet() do
14                 if statement → APIField ∈
                     suspicious_fields.get(level) then
15                     CI.get(level).add(statement)
```

3.2 Algorithm & Implementation

Algorithm 1 shows the key steps for detecting compatibility issues induced by API field evolution. It takes as input the source code of the app *Code* and the Android API differences report *APIDR*. First, it extracts the minimum, maximum and target SDK version information as API_{min}, API_{max} and API_{target} respectively (line 1 - 3) and uses them to identify the suspicious API fields (line 4 - 8). Then, it collects the API field usages in the source code as *APIUsages* using *collectAPIFieldUsages(code)* (line 9). *APIUsages* keeps the class, method, statement, the API field and the API level constraint information of the API field usages. Finally it checks whether any API field usage is suspicious for any API level and the API level does not satisfy the constraint, it reports the API field usage as a compatibility issue (line 10 - 15).

To get the classes and fields of API changes involved in an API update, API Difference Collector scrapes the Android API changes report webpage using Beautiful Soup [4]. Beautiful Soup is a Python library for pulling data out of HTML files, providing an idiomatic way of navigating, searching, and modifying the parse tree.

We implemented the API Field Usage Locator using Soot [28], which is a Java framework to analyze, instrument, optimize and visualize Java and Android applications. FCID leverages Soot's [28] program dependence graph and call graph APIs to obtain the intra- and inter-procedural slices. In practice, the correctness of slicing results highly hinges on the quality of the statically constructed call graphs and program dependence graphs. It is well-known that statically constructing precise and sound call graphs and program

Table 1: Comparison between FCID and Lint in detecting API field compatibility issues.

Techniques	True Positive	False Positive	Precision Score	Avg Time Cost (s)
FCID	213	80	72.70%	34.27
Lint	16	46	25.81%	49.82

dependence graphs for Java programs is challenging due to the language features such as dynamic method dispatching and reflection. For this reason, FCID may report false positives when analyzing some apps.

3.3 Evaluation of FCID

To evaluate the effectiveness of FCID, we selected all the 2,965 real-world Android apps from F-Droid [6], which is a software repository for open-source Android apps. We decompiled APK files from F-Droid to evaluate FCID because most apps under analysis do not have source code available.

We used Jadx [10], a Dex to Java decompiler, to extract source code from the apk files. Since Jadx failed to extract source code for 108 apps, we applied FCID to the remaining 2,857 apps for our evaluation.

As a result, FCID reported 9,391 API field compatibility issues, with 39.16% of the apps having at least one compatibility issue. Since manually analyzing all the real-world apps is prohibitively expensive, we manually analyzed the 105 apps, which were also used for our study in Section 4.2.2, to validate the reported compatibility issues. Table 1 shows the number of true positives, false positives, and precision score for the 105 apps. Specifically, FCID reported a total of 293 compatibility issues, where 213 are true positives, with a precision score of 72.70%. Since the actual total number of compatibility issues in these apps is unknown, we are not able to calculate the recall.

We further compared our tool with Lint [17], which is an Android Development Tools (ADT) plugin that analyzes code bases for potential problems including compatibility issues. We applied Lint to all the 105 apps and found 62 API field compatibility issues, in which 16 are true positives, 46 are false positives. Lint missed 197 true positives that were detected by FCID. The reason behind Lint's under-performance is because it does not analyze methods under the annotation `@TargetApi` or `SuppressLint`. For example, app `PassAndroid` utilizes the mentioned annotation in the method `doPrint` in class `PrintHelper`. The method uses `android.print.PrintManager` which was introduced only in API level 19.

Table 1 also shows the time cost of FCID and Lint. For a fair comparison, the time cost of FCID reported in the table does not count the time for parsing API differences reports, because Lint uses the available APIs through Android Studio. Yet, we note initially FCID took 131 seconds on average to parse an API differences reports. These API differences are then re-used across different apps to analyze each app for compatibility issue detection. As a result, on average it took 34.27 seconds for FCID to analyze an app versus 49.82 seconds for Lint.

4 STUDY METHODOLOGY

This section presents our research questions, followed by a description of how we collected data to study these questions.

4.1 Research Questions

This study aims to answer the following research questions:

- *RQ0: How frequently do Android API fields evolve?*
  This question aims to investigate how fast API fields evolve and its result motivates the following research question.
- *RQ1: How does Android API field evolution affect API compatibility in real-world apps?*
  This question aims to understand the relationship between API field evolution and the API field compatibility issues.
- *RQ2: How do Android API field compatibility issues get handled?*
  This question aims to study the different ways of handling API field compatibility issues, and how they are used in various scenarios in practice.
- *RQ3: How long does it take to fix an Android API field compatibility issue?*
  This question aims to understand the cost of detecting and fixing API field compatibility issues.

4.2 Data Collection

To answer research question RQ0, all API differences from Android API differences reports published by Google were collected; and to answer RQ1, RQ2 and RQ3, compatibility issues and related data from repositories of real-world open-source Android apps were collected.

4.2.1 Collection of API differences. We collected all added, removed, and changed APIs (fields and methods) from Android API differences reports [18]. The API Difference Collector of FCID as described in Section 3.1.1 was used to collect them. We also collected the class or package information of the changed fields and methods to analyze the correlation between API field updates and API method updates.

4.2.2 Collection of Compatibility Issues. To perform a large-scale study, we collected API field compatibility issues and relevant information from real-world open-source Android apps that are available on both F-Droid [6] and GitHub [7]. We selected apps using the following two criteria:

- each app has over 50 commits;
- each app has over five tags created after the release of Android API 19.

Applying these criteria, we selected a total of 681 apps, out of which we randomly selected a subset of 105 Android apps, which has a total of 11,098 tags and covers a total of 22 categories.

To answer RQ1 and RQ2, we crawled all the available tags of each app from their GitHub repositories using GitHub Rest API [8]. We chose tags instead of commits as tags are used for released versions of apps. In contrary, commits are used for various sizes of change sets in the app. To answer RQ3, we applied commit-level analysis to collect how much time was taken to fix the API field compatibility issues. Using tags to compute the time is not precise, because issues

Table 2: Evolution of API fields versus API methods in the Android API

API level	Release date	API Fields				API Methods			
		ADD	CHG	REM	Total	ADD	CHG	REM	Total
16	Jul 9, 2012	171	46	4	221	381	151	20	552
17	Nov 13, 2012	155	69	3	227	150	37	19	206
18	Jul 24, 2013	131	25	1	157	155	44	4	203
19	Oct 31, 2013	391	6	0	397	268	26	5	299
20	Jun 25, 2014	45	2	0	47	32	5	1	38
21	Nov 12, 2014	1150	75	2	1227	770	117	29	916
22	Mar 9, 2015	53	1	13	67	73	128	3	204
23	Oct 5, 2015	466	89	83	638	541	132	38	711
24	Aug 22, 2016	585	31	5	621	877	127	13	1017
25	Oct 4, 2016	53	0	0	53	50	1	0	51
26	Aug 21, 2017	572	69	0	641	795	139	18	952
27	Dec 5, 2017	42	2	0	44	206	176	16	398
28	Aug 6, 2018	469	272	8	749	483	673	169	1325
29	Sep 3, 2019	539	503	12	1054	793	1443	14	2250
30	Sep 8, 2020	752	128	8	888	580	295	0	875

are fixed at commit level while using tags may over-approximate the results.

This work focused on API fields that are available in the API documentation. These fields do not have setters and getters, and are directly accessed by the developers. The field evolution problem can conceptually reduce to the method evolution problem if all field accesses are made via setters and getters, but it may not be a good design as most of the API fields are static or constant variables.

We applied FCID and ACID [37] to all the tags of the selected Android apps to collect API field compatibility issues and API method compatibility issues, respectively. ACID is the state-of-the-art tool for detecting API method compatibility issues. Both tool might report some false positive results. When evaluating FCID, we manually investigated the 105 apps and calculated the false-positive rates. We estimate that the false-positive rates in the findings reported in Section 5 remain similar.

We analyzed these app repositories to find out when compatibility issues were introduced, and how and when they were addressed by the developers during the app evolution.

5 RESULTS AND ANALYSIS

5.1 RQ0: How frequently do Android API fields evolve?

McDonnell et al. [39] conducted an empirical study on the evolution of Android APIs and found out: in each month, on average 44 methods are changed, 11 methods are added, 51 fields are changed, 9 fields are added, and less than one method or field are removed. That study involved APIs at levels 3 to 15. Here we focused on the APIs at levels 16 to 30 to understand how API fields evolve more recently.

We collected added, changed, and removed API fields and methods for each Android level using API differences reports as explained in Section 3.1.1. When a new API version is released, Google provides an API difference report documenting all API changes in each release. We built a tool that extracts API change information from the HTML file. Table 2 shows the evolution of API fields versus the evolution of API methods from API level 16 to level 30. In the table, we report the added, changed and removed API fields ADD, CHG and REM respectively. Here, the number of additions is higher than changes and removals. Moreover, the total number of API field differences is comparable to the total number of API method differences.

We also computed how many APIs are updated in each month on average, same as McDonnell et al. [39] to measure the rate of Android API evolution, using the following formula:

$$avg.updateRate = \frac{\sum_{releases} APIupdates}{Months} \quad (1)$$

On average 53.09 fields were added, 12.55 fields were changed, and 1.32 fields were removed in each month; on the other hand, 58.61, 33.28 and 3.32 methods were added, changed and removed, respectively. The numbers for the last five API levels are 55.21, 22.65 and 0.65 for added, changed and removed API fields, respectively, versus 66.44, 63.40 and 5.045 for added, changed and removed API methods, respectively. The rate for added, changed and removed API fields is 29.67% smaller than the rate for added, changed and removed API methods. Yet, if we only consider compatibility relevant changes, i.e., the added and removed API fields and API methods, then rate for API fields is only 12.14% less than the rate of API methods.

We also analyzed the correlation between API field updates and API method updates. We considered a field update and a method update are correlated, when they are updated in the same API update and they belong to the same class. Suppose, API field AF and API method AM belong to the same class and are both updated in API update 30, they will be considered correlated. From the updates shown in Table 2, 63.28% of the method updates and 71.14% field updates are correlated. It is not surprising as a new API field is often introduced with new method(s). While analyzing in detail API method compatibility issues may discover the correlated API field compatibility issues, it leaves many API field compatibility

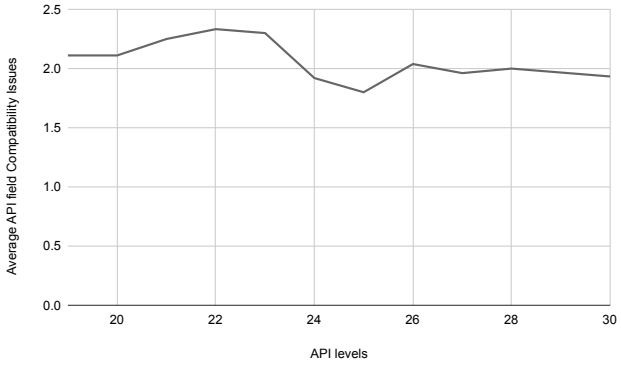

Figure 2: API field compatibility issues at each API level.

issues un-detected. Thus, API field specific analysis like FCID is demanded to detect and analyze the API field compatibility issues.

> ***Finding 0****: (1) API fields evolve frequently, and its evolution rate is only 12.14% less than API method evolution (when compatibility relevant evolution is considered); (2) Many API field compatibility issues may not be detected by existing API method level analysis, and API field specific analysis is demanded.*

As API fields are evolving frequently like the API methods and API field evolution has rarely been studied before, it motivates us to further study the following research questions RQ1-RQ3, to understand how API field evolution impacts API compatibility and how the compatibility issues are handled in real-world apps compared to API method evolution.

5.2 RQ1: How does Android API field evolution affect API compatibility in real-world apps?

To answer this question, we went through 11098 tags of 105 apps to examine how compatibility issues change during API evolution. In particular, we collected the potential compatibility issues induced by API field evolution in all the tags and summarized them to see how many compatibility issues were introduced and addressed during API evolution. We started from the API level 19, since no selected apps were released before the API level 19.

FCID detected a total of 24497 API field compatibility issues over all the tags. 1733 of them are unique, since one API compatibility issue may exist in multiple tags of an app. Figure 2 shows the average compatibility issues induced by API field evolution over the releases of these API levels. On average, there are two compatibility issues per app in each tag. Initially released tags are more bug prone than the later ones. For example, the number of issues increases at API levels 22 and 23, because some apps were initially released there. Also, we see the decreasing of issues at API levels 25 and 27 as the added, changed and removed API fields are few for these two API levels.

We also investigated the correlation between the API field compatibility issues and API method compatibility issues. Here we try to study if correlated API fields and methods that are used in the same class of the app, that induced compatibility issues in a similar app. For example, the AmazeFileManager [1] app, the developers used the fingerprint package `android.hardware.fingerprint` (introduced in API level 23) in FingerprintHandler class. They used the API methods 'authenticate' as well as `FINGERPRINT_ACQUIRED_GOOD` API field. We mark these usages as correlated API compatibility issues. We found that 56.70% API field compatibility issues are related to some API method compatibility issues in the same app, which left 43.30% API field compatibility issues to be analyzed separately. This shows the urgency of research on API field compatibility issues.

Table 3: Evolution of compatibility issues in Wikipedia app.

API level	Tags on GitHub	Compatibility issues	
		API fields	API methods
23	r/2.1.131-r-2015-09-28	2	6
24	r/2.3.149-r-2016-07-28	1	4
25	r/2.4.183-r-2016-12-08	0	5
26	beta/2.6.198-beta-2017-06-09	3	5
27	r/2.7.221-r-2017-12-08	1	4
28	r/2.7.239-r-2018-07-31	2	5
29	r/2.7.50291-r-2019-08-28	2	8
30	r/2.7.50330-r-2020-08-04	1	6

To better understand the effect, we further analyzed Wikipedia [16], which is the most downloaded app we analyzed for this study. Table 3 shows the results of Wikipedia app. With API level 23, the app had a total of eight compatibility issues, two of which were induced by API field evolution. The API level 24 was released on August 22, 2016 and by that time developers addressed one API field compatibility issue. Two other compatibility issues were fixed by the release of API level 25.

When API level 26 was released, there existed three API field compatibility issues. We investigated the origin and the handling of these issues and found that, the fields were not removed from the code, instead they were handled using the SDK version check. The numbers of fixes over the years for both API field and method induced compatibility issues shows that, the developers are giving the same priority to the API field's evolution induced compatibility issues.

From 24497 detected API field compatibility issues in all the tags of 105 selected apps, 15106 issues were fixed over the time. Although, the number of compatibility issues induced by API field's evolution might be smaller than the compatibility issues induced by API methods, API field compatibility issues were hampering the app quality and the developers had tried to address them whenever they find the issue.

> ***Finding 1****: (1) On average, there are two API field compatibility issues per app in each tag; (2) these issues hamper the app quality, which shows the importance of more analysis on API field compatibility issues.*

Table 4: Handling API compatibility issues.

Compatibility Issues Induced by	if-then	if-then-else	@RequiresApi
API fields	42.55%	30.40%	27.05%
API methods	43.17%	35.97%	20.86%

Table 5: Time to fix API compatibility issues.

Compatibility Issues Issues Induced by	Lowest (days)	Highest (days)	Average time (days)
API fields	11	291	92.37
API methods	9.00	317.00	48.33

5.3 RQ2: How do Android API field compatibility issues get handled?

To answer this question, we manually tracked the evolution of 1733 unique API field compatibility issues and 2409 unique API method compatibility issues and investigated the changes made by the developers whenever these issues were handled. As discussed in Section 2, developers use "if-then", "if-then-else", "@RequiresApi" annotation to perform API level checks in Android.

Hao et al. [49] recently conducted a large-scale study on the real-world Android market apps to shed light on the practice of developers in handling evolution-induced API compatibility issues. But they only considered the "if-then" or "if-then-else" cases. Here we try to investigate how the "@RequiresApi" annotations are used to address both API field compatibility issues and API method compatibility issues.

Table 4 shows the percentage of the cases handled by performing each type of checks. The "if-then" check means only the SDK version check and the "if-then-else" check means the SDK version check with a replacement API that the app will use for the other API levels. On the other hand, the "@RequiresApi" means the handling of compatibility issues using the annotation.

According to Table 4, the ratio of the "if-then" check is higher for addressing both API fields (42.55%) and API methods (43.17%) compatibility issues. Developers use it to initially handle a compatibility issue. Although Google recommends the replacement check for most APIs, the percentage of replacement check is relatively lower than the only API level check in the real-world apps. The use of "@RequiresApi" to address the API field compatibility issues is 27.05%, which is surprisingly higher than the 20.86% than the API methods.

We have also made the following observations on handling API field compatibility issues:

- Developers initially use only SDK version check to address API field compatibility issues. Later they apply replacement APIs to perform the similar functionality in other API levels that are restricted by the SDK version check.
- The "if-then" and "if-then-else" API level checks are more popular in handling forward compatibility issue. When one API is deprecated or removed, developers use these two checks to handle the incompatible API usages.
- The use of "@RequiresApi" annotation is more popular than the other two checks to address the backward compatibility issues. When a new feature is introduced in an API level, developers use these annotations to the whole class or to some specific methods so that these classes or methods can only be used in the specific API level or higher. For example, fingerprint package `android.hardware.fingerprint` was introduced in API level 23. When developers implement the fingerprint feature in an app, they usually use the @RequiresApi to restrict the implementation of that package so that devices with API level less than 23 cannot run that part.

> ***Finding 2**: (1) Most of API field compatibility issues are handled by developers using "if-then"; (2) The use of "@RequiresApi" is surprisingly high in addressing API field compatibility issues; (3) The "if-then" and "if-then-else" API level checks are more popular when addressing forward compatibility issues, while "@RequiresApi" is more popular when addressing backward compatibility issues.*

5.4 RQ3: How long does it take to fix an Android API field compatibility issue?

To answer this question, for each identified API compatibility issue, we collected the date of the commit where the issue was introduced as well as the date of the commit where the issue was fixed. In this experiment, we excluded the API field and method compatibility issues that are never fixed. Surprisingly, we find 38 such compatibility issues (from apps with minimum 20 commits after that) that were introduced more than a year ago. 13 of these issues were induced by API field evolution and 25 were induced by API method evolution. From the 1733 API field compatibility issues and 2409 API method compatibility issues, we got 196 unique API fields and 244 API methods which are recurring throughout the apps and each of them were fixed in any of the app's evolution.

Table 5 shows the lowest, highest and average time spent by developers to fix these issues. According to the results, the average time to fix an API field compatibility issue (92.37 days) is twice the time to fix an API method compatibility issue. This could happen for many reasons: there is less tool support for detecting/fixing API field compatibility issues, or developers put the fix of API field compatibility issues in a lower priority.

To better understand this, we analyzed some of the issues and found that, developers fixed the issues as soon as they were informed about the issues (the creation date of the issue). For example, AnkiAndroid [3] has an issue (#7135) [2], which was tagged as bug and priority high. The issue was first introduced in the code on Aug 19, 2020. After 30 days, they were informed about the issue and the developers fixed it within 2 days. Another issue (#5507) [3] was fixed by developers within a day after creating the issue, while the issue was introduced to the app 48 days ago. This indicates that the developers are concerned about the field evolution and its induced compatibility issues, and they fix these issues as soon as they find them.

[2]https://github.com/ankidroid/Anki-Android/issues/7135
[3]https://github.com/ankidroid/Anki-Android/issues/5507

***Finding 3**: On average, it takes more than three months to fix API field compatibility issues as there is little research and tool support for automated detection and fix of these issues.*

6 IMPLICATIONS

Our experimental study yields several interesting findings and reveals the importance of incorporating API fields in future studies on Android API evolution. We believe the findings of our study will help developers and researchers in better understanding, detecting, and handling Android compatibility issues induced by API field evolution.

6.1 For researchers

Compatibility Issue Detection. Researchers have recently proposed several techniques for detecting and fixing API-related compatibility issues in Android apps [26, 27, 30, 36, 47–49]. However, all these techniques are targeted at API methods. While it seems that some of these techniques might work on API fields after some modifications, no existing technique is working directly on API fields to the best of our knowledge.

In RQ3, the findings shows that, developers fix API field compatibility issues as soon as they find these issues but it takes time to discover them as there is little research and tool support for automated detection of these issues. Researchers are recommended to incorporate API fields in future studies to invent more and better techniques and tools like FCID for automated detection of API field compatibility issues.

Automated API Usage Update. Because of the fragmentation of Android, upgrading usages of deprecated Android APIs has become a top priority. Several studies have suggested automated approaches to updating Android API usages to assist developers [23, 24, 46]. Our findings of RQ2 on how developers handle API compatibility issues provide some insights on improving existing approaches in automated updating of API fields. Also, the findings provide some guidelines on the development of techniques to automatically handle API field compatibility issues by large.

API Evolution Study. In this study we investigate added, changed and removed API fields. While most deprecated APIs come with replacement messages suggesting alternatives, we have no evidence that the proposed alternatives are appropriate for app developers and the scenarios in which they used the deprecated APIs.

Also, the added APIs that could induce backward compatibility issues do not provide any alternatives that can be used in the previous versions. Researchers can investigate them to propose recommendation tools that developers can use to get alternatives for different API levels.

6.2 For developers

This study shows that the compatibility issues induced by API field evolution are also important and should not be ignored. These issues hamper the quality of the apps, and may take quite much time to fix. Also, Google recommends the replacement check for most APIs, but in our study we find that only 30.40% API field compatibility issues and 35.97% API method compatibility issues were addressed using the replacement check, which needs to be improved in future.

Moreover, project managers may want to assign specific tasks of detecting and handling API field compatibility issues when developing Android apps. Since answers to the questions related to API field evolution are lesser compared to API method evolution on online platforms such as Stack Overflow, it is highly recommended that experienced developers participate more actively in answering these questions.

7 THREATS TO VALIDITY

In terms of external validity, we used 105 open-source Android apps from GitHub, but the findings may not generalize to a wider range of mobile apps. We believe that our findings provide useful insights into the co-evolution of APIs and dependent applications since we selected projects from various application categories. However, more extensive study is needed to further address this threat.

The primary threats to internal validity are the potential faults in the implementation of our algorithms for collecting data from Android API differences reports and the GitHub repositories. We controlled for this threat by testing our implementations on examples that we can manually verify.

Where threats to construct validity are concerned, both the tools we used to detect compatibility issues induced by API fields (FCID) and by API methods (ACID) may generate false positive results. Also we do not consider the API fields where their identifier was changed, for example, from final to non-final or non-final to final. As final API fields can not be updated, these could potentially induce compatibility issues in other API levels. FCID does not take into account such compatibility issues.

8 RELATED WORK

In this section, we discuss the related work on Android API evolution and analysis of Android API compatibility issues.

8.1 Android API Evolution

In recent years, researchers have recognized the importance of Android API evolution and have studied how it impacts Android app development. Linares et al. [33] looked at the relationship between change-proneness and error-proneness API usages and Android app ratings and discovered that successful apps use APIs that are less fault-prone and change-prone than unsuccessful apps. In another research, [34], they found that Android API evolution would result in further Stack Overflow discussions. Lamothe et al. [29] reviewed the state-of-the-art researches on Android API evolution and outlined known challenges as well as new challenges uncovered during the review.

To understand the relationship between API evolution and adoption, McDonnell et al. [39] performed an empirical analysis of the evolution of Android APIs using version history data from GitHub. The findings of this study show that API updates are more vulnerable to flaws and bugs than other types of changes. Mutchler et al. [40] tested applications designed for older Android versions on devices running newer Android versions to see if severe security problems could be caused.

Li et al. [31] investigated the evolution of inaccessible Android APIs, considering both internal and hidden APIs. They followed previous changes to the Android framework code base, to find those inaccessible APIs. Furthermore, Li et al. [32] looked at how often deprecated APIs are used in the creation of Android apps and classified them. They also looked at how developers reacted to API deprecations. Yang et al. [50] investigated the effects of Android updates on apps and presented statistical estimates of how much an app is affected and Mahmud et al. [38] investigated affected code components and presented regression test selection approach based on the affected code. The framework-specific exceptions caused by API evolution were examined by Fan et al. [22]. Zhong et al. [51] conducted an empirical study on API usages, with an emphasis on how different types of APIs are used. They tried to understand the roles of API fields and static API elements as a part of the study. Businge et al. [19–21] have also performed many empirical studies on the Eclipse platform's API evolution.

Our work differs from the aforementioned works in two ways. First, we focus on API field evolution and compatibility issues induced by incompatible API fields in Android apps, and second, we investigate the evolution of 105 popular open-source apps to know how these compatibility issues evolve along with the app and how the developers handle these issues.

8.2 Compatibility Analysis

Wei et al. [47] conducted research into the signs and causes of compatibility issues and proposed FicFinder, a static-analysis tool that conducts static code analysis based on a model that collects Android APIs as well as the context in which compatibility issues occur. These problem occur when an improper API is used in a defective software/hardware setting. CiD is a system proposed by Li et al. [30] that detects compatibility issues using an Android Lifecycle Modeling (ALM) and Android Use Extraction (AUE). ALM examines the Android framework's source code, while AUE uses conditional call-graphs to generate usage details.

Huang et al. [27] proposed a study in which they looked at official Android documents and system code to create a graph that could be used to capture control flow inconsistencies. He et al. [26] performed a comprehensive empirical study that found significant API changes between Android versions, with the Android Support Library (provided by the Android OS) supporting less than 23% of the changes, potentially causing compatibility issues. They also introduced IctApiFinder, a tool that identifies API compatibility problems caused by unsupported API.

Wei et al. [48] proposed PIVOT, an Android app that automatically learns API-device associations of FIC issues. It is static in nature, just like Cider, and the unsoundness causes some connections to be missed or produced. Furthermore, since the technique depends on current FIC problems during the learning process, it is likely that some of them may be missed. Scalabrino et al. [42, 43] proposed *Acryl*, a data-driven approach to detect compatibility issues in Android APIs by analyzing the changes in the history of Android APIs. Mahmud et al. [36] proposed ACID, which detects both API invocation and callback compatibility issues using Android differences. They leverage the Android API differences report and backward data-flow analysis in detecting these issues.

Compared to these compatibility issue detection techniques, which consider only API method compatibility issues, our work focuses on API field evolution and introduces an automated technique for detecting API field compatibility issues.

Xia et al. [49] conducted a large-scale investigation into how Android apps are currently dealing with evolution-induced compatibility issues. They proposed RAPID, an automated method that uses static analysis and machine learning techniques to decide whether or not a compatibility problem has been solved. They also ignored the compatibility issues caused by incompatible API fields.

Fazzini et al. [23] recently proposed AppEvolve that automatically updates deprecated APIs in the Android apps. AppEvolve gathers update examples from existing codebases, abstracts their variable references, rates them according to planned applicability, and then attempts to add each abstracted example to the target app one by one. It applies the code and validates it with further testing if an abstracted example suits the code.

Thung et al. [45] found flaws in AppEvolve, including its generalizability and its requirement that the target file be organized in the same way as code examples. Based on these findings, Haryono et al. [24] proposed CocciEvolve [24] and then AndroEvolve [25] to address these flaws. Thung et al. [46] proposed NEAT, a method for generating transformation rules that help developers replace deprecated APIs in Android APIs. They only considered transformation between two forms of API methods, while API fields were not analyzed. The findings in our work would help improve these automatic deprecated API update techniques regarding API fields.

9 CONCLUSION

This paper presented an empirical study on Android API field evolution and its induced compatibility issues. First, we studied the evolution of the API fields; second, we used open-source apps to study how API field evolution induces compatibility issues in real-world Android apps; third, we studied how developers handle API field compatibility issues and how much time it takes to fix these issues. To facilitate our study, we also introduced a novel technique to detect API field compatibility issues in Android apps. Our study derives several interesting findings which help us gain an in-depth understanding about API field evolution and its induced compatibility issues. It also indicates the lack of tool and experience on dealing with API field compatibility issues, and motivates the need of methods and techniques for analyzing API field evolution and its induced compatibility issues, which is a urgent issue to be addressed and would impact real-world Android app development of today and future. For future work, we plan to manually analyze the remaining apps to further investigate the accuracy of FCID, and conduct an extended experiment on API field evolution and its induced compatibility issues with all of the 681 apps we collected from F-Droid and GitHub.

ACKNOWLEDGMENTS

We thank the reviewers for their insightful comments and suggestions. This work was partially supported by the US National Science Foundation under Grant No. CCF-1659807.

REFERENCES

[1] August 2021. Amaze File Manager. https://github.com/TeamAmaze/AmazeFileManager.

[2] August 2021. Android Version History. https://en.wikipedia.org/wiki/Android_version_history.

[3] August 2021. Anki-Android. https://github.com/ankidroid/Anki-Android.

[4] August 2021. Beautiful Soup. https://www.crummy.com/software/BeautifulSoup/.

[5] August 2021. Differences Between minSdkVersion, maxSdkVersion, compileSdkVersion and targetSdkVersion. https://www.thedroidsonroids.com/blog/compile-min-max-and-target-sdk-versions.

[6] August 2021. F-Droid. https://f-droid.org.

[7] August 2021. GitHub. https://github.com.

[8] August 2021. GitHub REST API. https://docs.github.com/en/rest.

[9] August 2021. Google Play Store. https://play.google.com.

[10] August 2021. JADX. https://github.com/skylot/jadx.

[11] August 2021. Mobile Operating System Market Share Worldwide. https://gs.statcounter.com/os-market-share/mobile/worldwide.

[12] August 2021. Omni Notes. https://github.com/federicoiosue/Omni-Notes.

[13] August 2021. @RequiresApi. https://developer.android.com/reference/androidx/annotation/RequiresApi.

[14] August 2021. @TargetApi. https://developer.android.com/reference/android/annotation/TargetApi.

[15] August 2021. Uses of API Level in Android. https://developer.android.com/guide/topics/manifest/uses-sdk-element#uses.

[16] August 2021. Wikipedia Android app. https://github.com/wikimedia/apps-android-wikipedia.

[17] Android. 2019. *Android Lint.*

[18] Android. 2021. *Android API Differences Report.* https://developer.android.com/sdk/api_diff/30/changes/changes-summary

[19] John Businge, Alexander Serebrenik, and Mark van den Brand. 2012. Survival of Eclipse third-party plug-ins. In *2012 28th IEEE International Conference on Software Maintenance (ICSM)*. IEEE, 368–377.

[20] John Businge, Alexander Serebrenik, and Mark van den Brand. 2013. Analyzing the Eclipse API usage: Putting the developer in the loop. In *2013 17th European Conference on Software Maintenance and Reengineering*. IEEE, 37–46.

[21] John Businge, Alexander Serebrenik, and Mark GJ Van Den Brand. 2015. Eclipse API usage: the good and the bad. *Software Quality Journal* 23, 1 (2015), 107–141.

[22] Lingling Fan, Ting Su, Sen Chen, Guozhu Meng, Yang Liu, Lihua Xu, Geguang Pu, and Zhendong Su. 2018. Large-scale analysis of framework-specific exceptions in Android apps. In *2018 IEEE/ACM 40th International Conference on Software Engineering (ICSE)*. IEEE, 408–419.

[23] Mattia Fazzini, Qi Xin, and Alessandro Orso. 2019. Automated API-usage update for Android apps. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 204–215.

[24] Stefanus A Haryono, Ferdian Thung, Hong Jin Kang, Lucas Serrano, Gilles Muller, Julia Lawall, David Lo, and Lingxiao Jiang. 2020. Automatic Android deprecated-API usage update by learning from single updated example. In *Proceedings of the 28th International Conference on Program Comprehension*. 401–405.

[25] Stefanus A Haryono, Ferdian Thung, David Lo, Lingxiao Jiang, Julia Lawall, Hong Jin Kang, Lucas Serrano, and Gilles Muller. 2021. AndroEvolve: Automated Update for Android Deprecated-API Usages. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 1–4.

[26] Dongjie He, Lian Li, Lei Wang, Hengjie Zheng, Guangwei Li, and Jingling Xue. 2018. Understanding and detecting evolution-induced compatibility issues in Android apps. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 167–177.

[27] Huaxun Huang, Lili Wei, Yepang Liu, and Shing-Chi Cheung. 2018. Understanding and detecting callback compatibility issues for android applications. In *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. 532–542.

[28] Patrick Lam, Eric Bodden, Ondrej Lhoták, and Laurie Hendren. 2011. The Soot framework for Java program analysis: a retrospective. In *Cetus Users and Compiler Infastructure Workshop (CETUS 2011)*, Vol. 15. 35.

[29] Maxime Lamothe, Yann-Gaël Guéhéneuc, and Weiyi Shang. 2021. A systematic review of API evolution literature. *ACM Computing Surveys (CSUR)* 54, 8 (2021), 1–36.

[30] Li Li, Tegawendé F Bissyandé, Haoyu Wang, and Jacques Klein. 2018. Cid: Automating the detection of API-related compatibility issues in android apps. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*. 153–163.

[31] Li Li, Tegawendé F. Bissyandé, Yves Le Traon, and Jacques Klein. 2016. Accessing Inaccessible Android APIs: An Empirical Study. In *2016 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. 411–422. https://doi.org/10.1109/ICSME.2016.35

[32] Li Li, Jun Gao, Tegawendé F Bissyandé, Lei Ma, Xin Xia, and Jacques Klein. 2018. Characterising deprecated android APIs. In *Proceedings of the 15th International Conference on Mining Software Repositories*. 254–264.

[33] Mario Linares-Vásquez, Gabriele Bavota, Carlos Bernal-Cárdenas, Massimiliano Di Penta, Rocco Oliveto, and Denys Poshyvanyk. 2013. API change and fault proneness: a threat to the success of Android apps. In *Proceedings of the 2013 9th joint meeting on foundations of software engineering*. 477–487.

[34] Mario Linares-Vásquez, Gabriele Bavota, Massimiliano Di Penta, Rocco Oliveto, and Denys Poshyvanyk. 2014. How do API changes trigger stack overflow discussions? a study on the android sdk. In *proceedings of the 22nd International Conference on Program Comprehension*. 83–94.

[35] Pei Liu, Li Li, Yichun Yan, Mattia Fazzini, and John Grundy. 2021. Identifying and characterizing silently-evolved methods in the android API. In *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 308–317.

[36] Tarek Mahmud, Meiru Che, and Guowei Yang. 2021. Android Compatibility Issue Detection Using API Differences. In *2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. 480–490. https://doi.org/10.1109/SANER50967.2021.00051

[37] Tarek Mahmud, Meiru Che, and Guowei Yang. 2022. ACID: An API Compatibility Issue Detector for Android Apps. In *2022 IEEE/ACM 44th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE, 1–5.

[38] Tarek Mahmud, Mujahid Khan, Jihan Rouijel, Meiru Che, and Guowei Yang. 2021. API Change Impact Analysis for Android Apps. In *2021 IEEE 45th Annual Computers, Software, and Applications Conference (COMPSAC)*. 894–903. https://doi.org/10.1109/COMPSAC51774.2021.00122

[39] Tyler McDonnell, Baishakhi Ray, and Miryung Kim. 2013. An empirical study of API stability and adoption in the android ecosystem. In *2013 IEEE International Conference on Software Maintenance*. IEEE, 70–79.

[40] Patrick Mutchler, Yeganeh Safaei, Adam Doupé, and John Mitchell. 2016. Target fragmentation in Android apps. In *2016 IEEE Security and Privacy Workshops (SPW)*. IEEE, 204–213.

[41] Ben Popper. 2017. *Google announces over 2 billion monthly active devices on Android.* Retrieved November 25th, 2018 from https://www.theverge.com/2017/5/17/15654454/android-reaches-2-billion-monthly-active-users

[42] Simone Scalabrino, Gabriele Bavota, Mario Linares-Vásquez, Michele Lanza, and Rocco Oliveto. 2019. Data-driven solutions to detect API compatibility issues in android: an empirical study. In *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*. IEEE, 288–298.

[43] Simone Scalabrino, Gabriele Bavota, Mario Linares-Vásquez, Valentina Piantadosi, Michele Lanza, and Rocco Oliveto. 2020. API compatibility issues in Android: Causes and effectiveness of data-driven detection techniques. *Empirical Software Engineering* 25, 6 (2020), 5006–5046.

[44] Statista. May 2020. *Number of available applications in the Google Play Store from December 2009 to May 2020.* Retrieved June 2, 2020 from https://www.statista.com/statistics/266210/number-of-available-applications-in-the-google-play-store/

[45] Ferdian Thung, Stefanus A Haryono, Lucas Serrano, Gilles Muller, Julia Lawall, David Lo, and Lingxiao Jiang. 2020. Automated Deprecated-API Usage Update for Android Apps: How Far Are We?. In *2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 602–611.

[46] Ferdian Thung, Hong Jin Kang, Lingxiao Jiang, and David Lo. 2019. Towards generating transformation rules without examples for Android API replacement. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 213–217.

[47] L. Wei, Y. Liu, and S. Cheung. 2016. Taming Android fragmentation: Characterizing and detecting compatibility issues for Android apps. In *2016 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 226–237.

[48] L. Wei, Y. Liu, and S. Cheung. 2019. PIVOT: Learning API-Device Correlations to Facilitate Android Compatibility Issue Detection. In *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. 878–888.

[49] Hao Xia, Yuan Zhang, Yingtian Zhou, Xiaoting Chen, Yang Wang, Xiangyu Zhang, Shuaishuai Cui, Geng Hong, Xiaohan Zhang, Min Yang, et al. 2020. How Android developers handle evolution-induced API compatibility issues: a large-scale study. In *2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)*. IEEE, 886–898.

[50] Guowei Yang, Jeffrey Jones, Austin Moninger, and Meiru Che. 2018. How Do Android Operating System Updates Impact Apps?. In *Proceedings of the 5th International Conference on Mobile Software Engineering and Systems* (Gothenburg, Sweden) *(MOBILESoft '18)*. ACM, New York, NY, USA, 156–160. https://doi.org/10.1145/3197231.3197258

[51] Hao Zhong and Hong Mei. 2017. An empirical study on API usages. *IEEE Transactions on Software Engineering* 45, 4 (2017), 319–334.